

FlexiBooking

Admin UI / UX Gap Analysis Report

Prepared: April 28, 2026 | Scope: Admin-side UI/UX against §1.3–§1.9 + §1.10
Source: sg-booking-management plugin + FlexiBooking_Specifications_EN.docx

5	9	11	4
Critical violations	Functional gaps	UX pattern gaps	Missing entirely

Severity legend

CRITICAL	Spec requirement directly violated or the feature cannot work as specified.
FUNCTIONAL	Feature is partially built but a key functional element is absent or wrong.
UX	The behaviour is wrong for the admin context — wrong input pattern, missing label, or confusing presentation.
MISSING	A required admin view, field, or tracking mechanism is entirely absent from the codebase.

Feature-by-Feature Gap Analysis

Feature 1 — Shared Inventory Extras

Spec reference: §1.3

PARTIAL

■ [FUNCTIONAL] Split navigation between two separate admin pages

The Shared Extras feature lives under its own menu page (bm_shared_extras) while Features 2–7 are grouped under the Booking Features tab page. The specification defines all 7 as a coherent set. An admin must navigate between two disconnected areas to manage related features, which fragments the mental model and makes cross-feature configuration error-prone.

■ [MISSING] No consumption or reporting dashboard

The spec explicitly requires "centralised stock, cost and consumption tracking" for shared extras (§1.3, reporting note). The current UI has a "Usage" badge per row that shows only "—" by default and must be clicked individually to load. There is no aggregate view showing which services consumed how many units of a shared extra over any date range. This makes the feature analytically invisible and violates the spec's reporting requirement.

■ [UX] Import flow risks recreating the "three identical extras" anti-pattern

The "Import from Service" feature clones a local extra into a new global one, but explicitly states "The original local extra remains unchanged." No warning is shown to the admin that the original local extra may still be attached to that service, meaning the service could now have both a local copy AND a global shared extra. This is the exact duplication the spec warns against.

Feature 2 — Product as Extra of Another Product

Spec reference: §1.4

PARTIAL

■ [UX] Add-on list displays raw "Addon Svc ID" numbers instead of service names

When a parent service's add-ons are loaded, the table column reads "Addon Svc ID: 23" rather than the resolved service name (e.g. "Massage"). An admin cannot audit or manage add-on relationships without separately looking up service IDs. Every other service selector in the admin uses named dropdowns — this table should resolve and display the service name.

■ [FUNCTIONAL] No add-on availability or booking logic visible from the relationship form

The spec states: "The product-extra retains its own price, availability, optional calendar and booking logic." The add-on relationship form in the admin shows only price override and frontend visibility toggle. There is no way for the admin to see or verify the add-on's own availability rules, booking capacity, or calendar from the same screen without leaving to find the add-on's service page.

■ [MISSING] Order view has no upsell/add-on tracking

The spec requires tracking "base service + add-on purchased" to read upsell performance and attach rate (§1.10 reporting table). The current order listing and single order view have no column, badge, section, or indicator showing that an order included a product-as-extra purchase. The analytical tracking for this feature is entirely absent.

Feature 3 — Service Chaining (Mutual Exclusion)

Spec reference: §1.5

CRITICAL GAPS

■ [CRITICAL] "Sequential" chain type is not defined in the specification

The Chain Type dropdown presents two options: "Mutual Exclusion" and "Sequential". The specification §1.5 defines only one rule for service chaining: a binary mutual exclusion where selling one service blocks the other. "Sequential" has no definition in the spec, no documented behaviour, and introduces undefined logic. An admin selecting "Sequential" does not know what it does — or whether it does anything at all. This option must either be removed or formally specified and documented.

■ [UX] Chain list shows raw service IDs — "Svc #4 / Svc #7" — not names

The loaded chains table renders service references as "Svc #4" and "Svc #7" rather than their names. An admin managing chains for a catalogue with dozens of services cannot audit or maintain these rules without manually cross-referencing every service ID. The table must resolve and display service names.

■ [FUNCTIONAL] No validation preventing a service from being chained to itself

The UI allows Service A and Service B to be set to the same service in the chain creation form. A self-chain rule (a service that mutually excludes itself) would make that service impossible to book on any slot. Client-side or server-side validation must prevent self-chaining.

■ [UX] No chain participation indicator on the service listing page

The service listing page has no badge, icon, or column indicating whether a service participates in any chaining rule. An admin viewing the service list cannot determine which services have mutual exclusion constraints applied without navigating to the Booking Features page and loading the chains list separately.

Feature 4 — Shared Calendar and Availability (Resource Pools)

Spec reference: §1.6

PARTIAL

■ [UX] "Capacity Used" field label is semantically wrong

When linking a service to a resource pool, the input field is labeled "Capacity Used". This reads as a read-only metric ("how much capacity has already been used") rather than the configuration intent ("how many seats each booking of this service consumes from the pool"). The label must be changed to something unambiguous such as "Seats consumed per booking" or "Capacity per booking".

■ [FUNCTIONAL] No remaining capacity counter or date-filtered availability view

The pool listing displays "Total Cap." as a static number but provides no view of current remaining capacity per date or slot. The spec's canonical use case (three wine tastings sharing 10 seats) requires the admin to understand "8 seats remain after 2 bookings on date X." There is no such live or date-filtered capacity view in the admin pool management screen.

■ [UX] Inline "Link Service to Pool" form uses a raw number input for service ID

Inside the expanded pool row, the form for linking a service has a plain numeric input labeled "Service ID". This is inconsistent with the rest of the admin, which uses named dropdowns for all service selection. An admin must manually look up and type service IDs, which is error-prone and does not scale to real catalogue sizes.

Feature 5 — Services with Options (Variants)

Spec reference: §1.7

PARTIAL

■ [FUNCTIONAL] No UI signal that options share the base service availability — not their own

The spec constraint is explicit: "Options have different prices but share the same availability of the base service." The option set creation form and option value form contain no field, label, informational note, or visual confirmation that makes this availability relationship visible to the admin. A person configuring options has no indication whether each option has its own independent capacity or inherits the service's pool — a critical operational distinction.

■ [MISSING] Selected option is not tracked in orders or analytics

The spec §1.7 explicitly states: "From an analytics perspective it is important to save both the service sold and the option chosen, as purchasing behaviour differs significantly." The current order listing, single order view, and analytics page have no column, section, or filter for the option variant selected at booking time. The feature exists in configuration but its analytical output is entirely absent.

■ [UX] No reordering of option values within a set

Option values are displayed in creation order with no drag-to-reorder or up/down controls. The order in which options are presented to the end customer (e.g. "Light" before "Full") directly affects conversion flow. Admins cannot control presentation order without deleting and recreating values in the desired sequence.

Feature 6 — Virtual Services

Spec reference: §1.8

CRITICAL GAPS

■ [CRITICAL] Create form has a single "Linked Service" dropdown — conceptually incorrect

The "Create Virtual Service" form presents a single dropdown labeled "Linked Service". This implies a virtual service is linked to one real service, when the specification defines it as an entity that depends on multiple real component services simultaneously. The correct flow is: define a named virtual service, then attach two or more real component services to it. The current UI obscures this two-level model and forces admins to discover the "Components" sub-table as an undocumented second step.

■ [CRITICAL] VS listing has dual identity — "Service ID" column and "Components" button — unexplained

The virtual service listing shows both a "Service ID" column (the wrapper service record) and a "Components" button (the underlying real services). This duality is unexplained in the UI. An admin cannot tell why a virtual service has both a service_id and separate component services, or which one is the "real" reference. The UI must clearly communicate the two-layer model: the virtual entity vs. its component real services.

■ [UX] Component service input is a raw number field inside the expanded row

Adding a component to a virtual service requires typing a raw service ID number. Given that the virtual service concept is already the most cognitively demanding of the seven features, extra UI friction here is particularly harmful. This input must be a service name dropdown or search-as-you-type selector, consistent with all other service selection patterns in the admin.

■ [MISSING] No bidirectional availability propagation preview or status panel

The spec's key complexity for virtual services is bidirectional propagation: "virtual sold → all components close" AND "one component sold → virtual closes, other component stays open." There is no admin panel, diagram, or status view showing the current propagation state of a virtual service and its components for any given date/slot. Without this, an admin cannot verify the rule is functioning correctly or diagnose availability conflicts.

Feature 7 — Bundles and Combos

Spec reference: §1.9

CRITICAL GAPS

■ [CRITICAL] Bundle has no price field, no status toggle, and no booking path

The spec states: "The bundle must be a specific, recognisable, standalone sellable entity." The bundle creation form provides a name, description, and discount rule but no base price, no active/inactive status toggle, and no mechanism to surface the bundle as a bookable product on the frontend. A bundle with no commercial identity and no booking entry point cannot be sold. This is the most fundamental missing element for this feature.

■ [FUNCTIONAL] No total price calculation or post-discount price preview while building a bundle

The spec requires a bundle to be "priced as the sum of components OR with a discount." When adding services to a bundle via the Items sub-form, the admin cannot see the sum of component prices or the resulting post-discount bundle price. Admins must perform this arithmetic manually and have no in-UI confirmation that the discount has been applied correctly.

■ [UX] Add-item form uses a raw service ID number input

Adding a service to a bundle requires typing a "Service ID" manually in a plain number input. This is error-prone and inconsistent with service selector dropdowns used throughout the rest of the admin. All service selection in Booking Features inline forms must use the same named dropdown pattern as the main feature forms.

■ [MISSING] Order view has no "bundle sold + components allocated" record

The spec requires: "the system must record the sale of the bundle AND correctly allocate each included service." The current order listing and single order view have no bundle context: no way to identify that an order was a bundle purchase, and no fulfilment breakdown showing which component services were allocated. Bundle tracking in orders and analytics is entirely absent.

Cross-Cutting Gaps — Apply to All 7 Features

The following gaps are not tied to a single feature. They affect the architecture and usability of the entire Booking Features admin and must be addressed independently.

1. No Edit action on 5 of 6 feature types

The Booking Features tab page supports only Create and Delete for bundles, service chains, resource pools, virtual services, and service extras. There is no Edit/Update flow for any of these entities. Spec-compliant feature management requires full lifecycle support: Create, Read, Edit, Delete. Only Shared Extras (separate page) has an edit capability.

2. Analytics page is completely feature-blind

The analytics page (`booking-management-analytics.php`) has no awareness of any of the 7 features. Specification §1.10 defines seven distinct tracking dimensions, one per feature (shared extra consumption, upsell attach rate, resource saturation, capacity mix, option preference, virtual cannibalisation, bundle revenue). None of these dimensions are present in the current analytics or reporting views.

3. Service listing shows no feature-participation badges

The service listing page surfaces only a shared-extras count per service. There are no indicators for: chain participation (mutual exclusion rules), resource pool membership, virtual service component role, option set presence, or bundle membership. An admin auditing the catalogue cannot determine a service's feature relationships without leaving the listing and checking each feature tab individually.

4. Raw service ID number inputs used across all inline sub-forms

The inline "add item" forms inside expanded table rows — for bundle items, virtual service components, resource pool service links, and the chain listing display — all use plain numeric input fields for service selection. Every other service selector in the admin uses a named dropdown (e.g. the main feature form dropdowns, the add-service page, the order form). All six inline sub-forms must be updated to use the same named dropdown or search-as-you-type pattern.

5. No spec-mapped contextual help or field descriptions in Booking Features

None of the six Booking Features tabs include contextual help text explaining the conceptual difference between the features, which is the most critical admin usability risk. The spec §1.10 dedicates a full section to explaining why these scenarios must remain distinct. Without in-UI guidance, admins will misconfigure features — particularly confusing pools vs. chains, or options vs. extras.

Prioritised Remediation Checklist

Recommended fix order based on severity and dependency. Complete Critical items before moving to Functional, then UX and Missing reporting layers last.

P1	CRITICAL	F7	Add a Price field, Status (active/inactive) toggle, and a frontend booking path to the Bundle entity. Without these, bundles cannot be sold.
P2	CRITICAL	F3	Remove the "Sequential" chain type dropdown option. It is undefined in the spec. Only "Mutual Exclusion" must be offered.
P3	CRITICAL	F6	Redesign the Virtual Service create form: remove the single "Linked Service" dropdown. Make the flow explicit — name the VS first, then add 2+ component services. Clarify the "Service ID vs Components" dual identity in the listing.
P4	CRITICAL	F3	Add client-side/server-side validation preventing a service from being chained to itself.
P5	CRITICAL	F2/F7	Add bundle and upsell (product-as-extra) line items to the Order view and Single Order page so purchased add-ons and bundles are visible per order.
P6	FUNCTIONAL	F4	Rename "Capacity Used" field to "Seats consumed per booking" in the pool service link form.
P7	FUNCTIONAL	F5	Add an informational note to the Options admin: "All values in this option set share the base service's availability pool."
P8	FUNCTIONAL	ALL	Replace all raw numeric service ID inputs in inline sub-forms (bundle items, VS components, pool links) with the named service dropdown pattern.
P9	FUNCTIONAL	F4	Add a date-filtered remaining capacity view to the Resource Pool listing.
P10	FUNCTIONAL	F7	Show a computed total price (sum of components) and post-discount price in the bundle builder as services are added.
P11	UX	F2/F3	Resolve and display service names in the Service Extras add-on list and the Service Chains listing (replace "Addon Svc ID: 23" and "Svc #4" with actual names).
P12	UX	F3	Add a chain-participation badge to the service listing page for all chained services.
P13	UX	F5	Add drag-to-reorder or up/down controls for option values within an option set.
P14	UX	F6	Replace the VS component raw ID input with a named service dropdown.
P15	UX	F1	Consolidate Shared Extras navigation into the Booking Features tab page (or add a prominent cross-link).
P16	MISSING	F5/F7	Record the selected option variant on each order line and expose it in the order listing, single order view, and analytics.
P17	MISSING	F1	Build a shared extra consumption dashboard: total units used per service, per date range, with cost tracking.
P18	MISSING	F6	Build a virtual service availability status panel showing bidirectional propagation state per date/slot.
P19	MISSING	ALL	Add Edit support for: bundles, service chains, resource pools, virtual services, and service extras.

P20	MISSING	ALL	Add 7 feature-specific tracking dimensions to the analytics page as per §1.10 reporting table in the spec.
-----	---------	-----	--

End of report. All findings reference the FlexiBooking_Specifications_EN.docx (April 2026) and the sg-booking-management plugin codebase as analysed.